© Abdelouahab MOUSSAOUI

Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science

CHAPTER 3

Convolutional Neural Networks (CNNs)

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

"Convolutional networks: teaching machines to see one kernel at a time."

— Pr. Abdelouahab Moussaoui

Abstract

This chapter provides a comprehensive introduction to **Convolutional Neural Networks (CNNs)**, following established frameworks and definitions from leading educational resources. Starting from biological inspiration and historical developments, we systematically explore the fundamental building blocks: convolution, padding, stride, pooling, and the typical architecture of a CNN. We then dive into major architectures that have shaped the field, from LeNet-5 to EfficientNet, highlighting their innovations and trade-offs. Practical implementation aspects are covered through a complete TensorFlow/Keras example on CIFAR-10, including data augmentation and regularization, as presented in standard course materials. By the end of this chapter, you will have a solid understanding of how CNNs work, how to design them, and how to apply them to real-world problems.

Contents

Preamble 5

1 Fundamental Principles of CNNs 6
- 1.1 Three Key Ideas . . . 6
- 1.2 Hierarchical Feature Learning . . . 6

2 Convolution 6
- 2.1 Architecture of a traditional CNN . . . 7
- 2.2 The Convolution Operation . . . 7
- 2.3 Filter Hyperparameters: Stride and Padding . . . 7
  - 2.3.1 Role of Stride . . . 8
  - 2.3.2 Role of Padding . . . 8
  - 2.3.3 Effect of Stride and Padding . . . 8
- 2.4 Depthwise Separable Convolution . . . 9
  - 2.4.1 Depthwise Convolution . . . 10
  - 2.4.2 Pointwise Convolution . . . 10
- 2.5 Activation Functions in Convolutional Layers . . . 11
  - 2.5.1 Why Nonlinearity? . . . 11
  - 2.5.2 Rectified Linear Unit (ReLU) . . . 11

3 Pooling Layers 12
- 3.1 Max Pooling . . . 12

4 Flatten Operation 12

5 Fully Connected Layers 13

6 CNN Architecture Recap 14

7 Major CNN Architectures 15
- 7.1 Evolution of CNN Architectures . . . 15
- 7.2 LeNet-5 (1998) . . . 16
- 7.3 AlexNet and the ImageNet Revolution (2012) . . . 16
- 7.4 VGGNet (2014) . . . 17
- 7.5 GoogLeNet / Inception v1 (2014) . . . 18
- 7.6 ResNet (2015) . . . 18
  - 7.6.1 ResNet-152 (152-layer version, 2015) . . . 18
  - 7.6.2 ResNet-50 (lightweight 50-layer version, 2015) . . . 19
- 7.7 DenseNet (2016) . . . 20
- 7.8 MobileNet (2017) . . . 21
- 7.9 EfficientNet (2019) . . . 22

8 Hands-On Example: CNN for CIFAR-10 Classification **23**
8.1 Dataset Description 23
8.2 Data Loading and Preprocessing 23
8.3 Data Augmentation 24
8.4 Model Definition 24
8.5 Compilation and Training 24
8.6 Evaluation 25

9 Applications of Convolutional Neural Networks **25**
9.1 Object Detection 25
9.2 Image Segmentation 25

Conclusion **26**

Preamble

About this Chapter

This chapter is part of the Deep Learning course for 4th year engineering students specializing in Artificial Intelligence. It focuses exclusively on Convolutional Neural Networks, building upon the foundational concepts covered in Chapter 2 (Introduction to Deep Neural Networks). The definitions and formulas presented draw upon widely adopted conventions and formulas from the deep learning literature. [?].

Prerequisites

- Solid understanding of backpropagation, activation functions, and optimization (Chapter 2)
- Familiarity with Python and TensorFlow/Keras basics
- Basic linear algebra and calculus

Learning Objectives

- Understand the motivation and history behind CNNs
- Master the mechanics of convolution, padding, stride, and pooling with precise formulas
- Analyze and compare major CNN architectures
- Implement CNNs using TensorFlow/Keras for image classification tasks
- Apply data augmentation and regularization to improve generalization

1 Fundamental Principles of CNNs

Convolutional Neural Networks (CNNs) are a specialized class of deep neural networks designed to process data with a grid-like topology, such as images (2D grids of pixels) or time series (1D grids of samples). Inspired by the organization of the animal visual cortex, CNNs exploit the spatial structure of the input to learn rich, hierarchical representations with far fewer parameters than fully connected networks. Their effectiveness stems from three fundamental principles: sparse connectivity, parameter sharing, and equivariance to translation. These principles not only make CNNs computationally efficient but also imbue them with built-in invariance to the location of features, which is crucial for tasks like object recognition.

1.1 Three Key Ideas

Definition 1.1 : Sparse Connectivity

Each neuron in a convolutional layer connects only to a small region of the input (local receptive field), unlike fully connected layers where every neuron connects to all inputs. This dramatically reduces parameters.

Definition 1.2 : Parameter Sharing

The same set of weights (filter/kernel) is applied across different spatial locations of the input. This enforces translation invariance and further reduces parameters.

Definition 1.3 : Equivariance to Translation

If the input shifts, the feature map shifts correspondingly. This property helps CNNs detect features regardless of their position.

1.2 Hierarchical Feature Learning

CNNs automatically learn a hierarchy of features:

- **Low-level layers**: edges, colors, textures
- **Mid-level layers**: parts of objects (eyes, wheels, corners)
- **High-level layers**: complete objects or concepts

2 Convolution

The convolution operation is the core building block of a convolutional neural network. It involves sliding a small array of weights, called a filter or kernel, across the input image (or feature map) and computing the dot product between the filter and the corresponding local region.

2.1 Architecture of a traditional CNN

Convolutional neural networks, also known as CNNs, are a specific type of neural networks that are generally composed of the following layers:

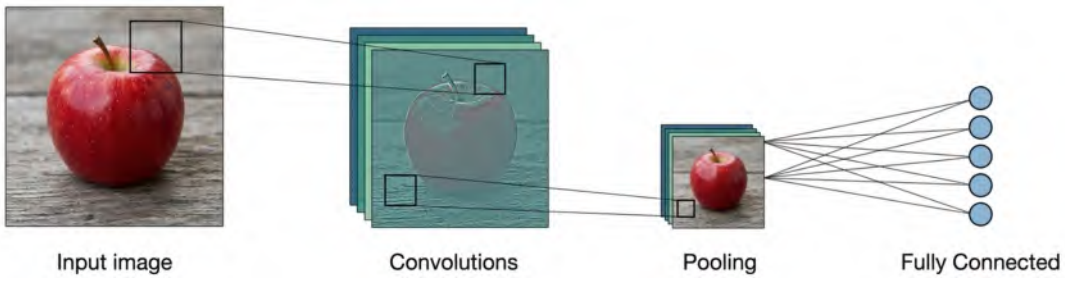


Figure 1: The convolution layer and the pooling layer can be fine-tuned with respect to hyperparameters that are described in the next sections.

2.2 The Convolution Operation

Definition 2.1 : 2D Convolution

For an input I of size $H \times W$ (or volume $H \times W \times C$) and a filter (kernel) K of size $F \times F \times C$, the convolution output O (feature map) at spatial position (i, j) is:

$$O(i,j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \sum_{c=1}^{C} I(i+m, j+n, c) \cdot K(m,n,c) + b$$

where b is a bias term (usually one per filter).

■ **Note** — A filter of size $F \times F$ applied to an input with C channels is actually a volume $F \times F \times C$. After convolution with one filter, we obtain a 2D feature map of size $O \times O \times 1$. If we use K filters, the output becomes $O \times O \times K$.

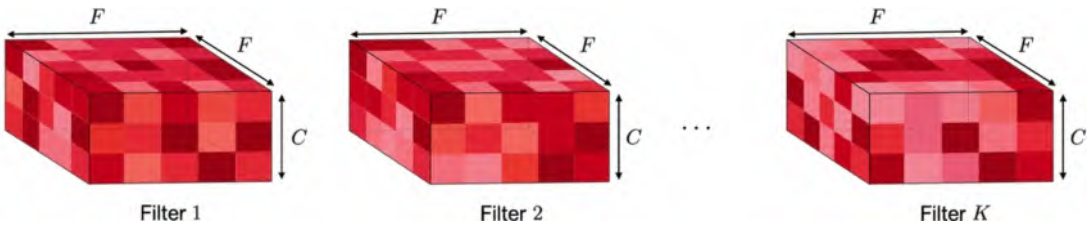


Figure 2: Structural organization of convolution kernels. A set of K volumetric filters where F denotes the spatial dimensions and C represents the number of input channels..

2.3 Filter Hyperparameters: Stride and Padding

2.3.1 Role of Stride

- **Stride** determines the step size by which the filter moves across the input. - A stride of 1 produces a dense output that retains fine spatial details. - Larger strides (e.g., 2) skip pixels, resulting in a smaller output and reduced computation, but may lose resolution.

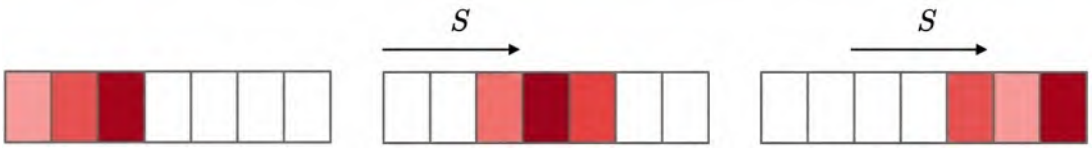


Figure 3: Visualization of the stride (S) parameter. The arrows indicate the displacement of the filter across the input vector after each convolution operation.

2.3.2 Role of Padding

- **Padding** adds extra values (typically zeros) around the input borders. - It allows the filter to process edge pixels more effectively, avoiding information loss at the boundaries. - Padding also helps control the output size: with appropriate padding, the output can be kept the same size as the input (when stride = 1). - Common padding modes include *valid* (no padding), *same* (output size equals input size for stride 1), and *full* (maximum padding).

Three padding modes are commonly used:

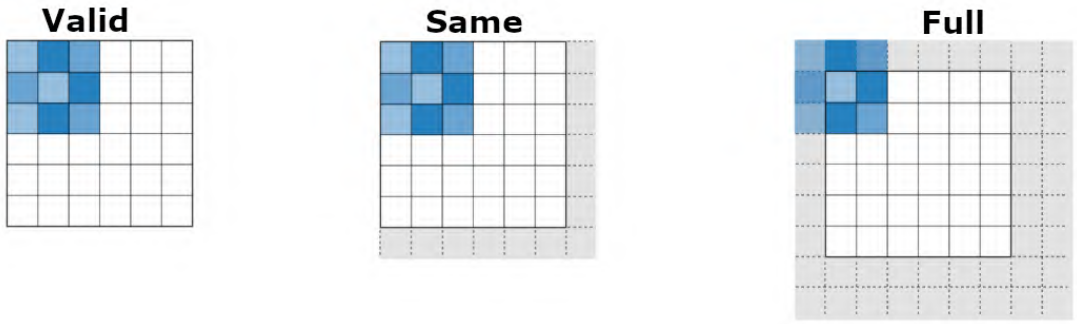


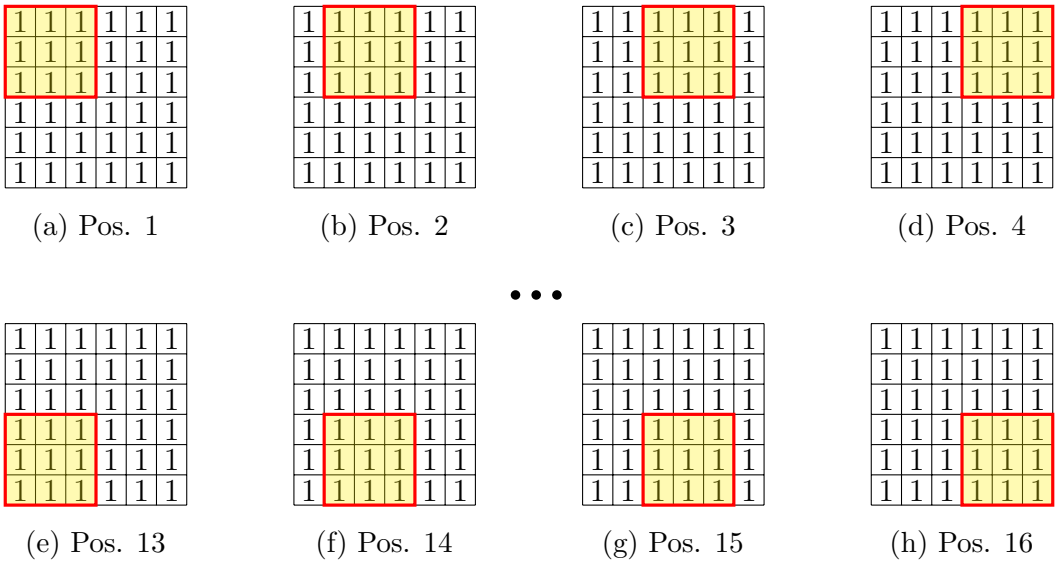
Figure 4: Comparison of common padding strategies in 2D convolutions.

2.3.3 Effect of Stride and Padding

Common padding modes:

- **Valid padding** ($P = 0$): no padding; the output size shrinks, and the last convolution may be dropped if dimensions don't align.
- **Same padding**: pads so that the output size equals $\lceil I/S \rceil$ (when $S = 1$, output size equals input size).

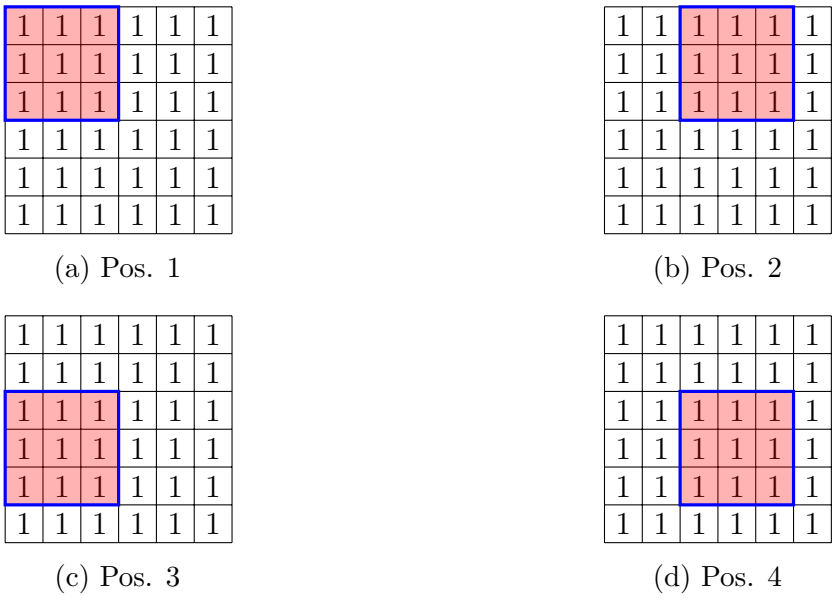
- **Full padding**: adds maximum padding so that every input element is visited the same number of times; rarely used in deep networks.



(a) Pos. 1 (b) Pos. 2 (c) Pos. 3 (d) Pos. 4

(e) Pos. 13 (f) Pos. 14 (g) Pos. 15 (h) Pos. 16

Figure 5: Filter shift with stride = 1 (16 positions, only rows 1 and 4 are shown).



(a) Pos. 1 (b) Pos. 2

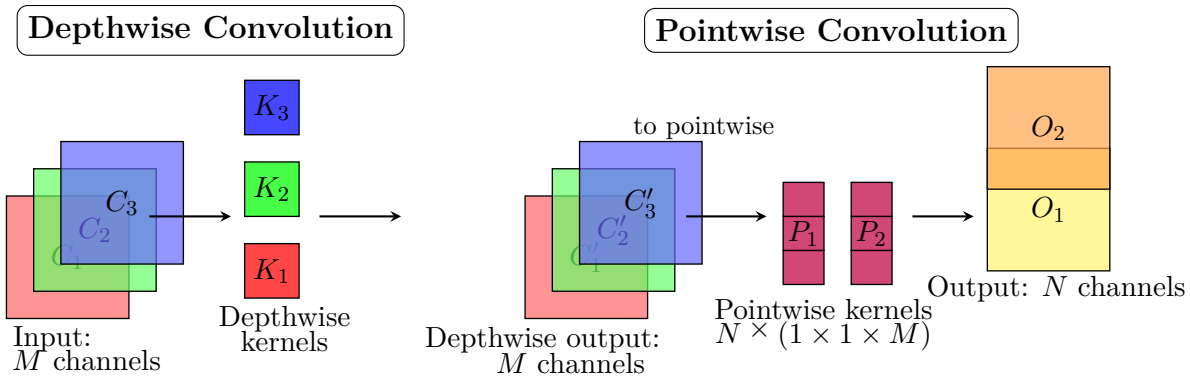
(c) Pos. 3 (d) Pos. 4

Figure 6: Filter movement with stride = 2 (4 positions). The rectangle has a blue outline and a transparent red background.

2.4 Depthwise Separable Convolution

Depthwise separable convolutions factorize the standard convolution into two distinct operations: depthwise convolution and pointwise convolution. This factorization decouples

spatial filtering from channel mixing, resulting in a significant reduction in computational cost.



Depthwise: each channel filtered independently with $D_K \times D_K$ kernels.
Pointwise: 1×1 convolutions mix channels to produce new representations.

Figure 7: Depthwise separable convolution factorized into two distinct stages: depthwise convolution (left) applies separate spatial filters to each input channel, preserving channel count; pointwise convolution (right) then applies 1×1 filters to combine channels, producing the desired number of output channels. This separation drastically reduces computational cost compared to standard convolution.

2.4.1 Depthwise Convolution

In the depthwise convolution step, instead of applying N filters that operate on all M channels simultaneously, we apply a single filter per input channel. Specifically, for each input channel $m \in \{1, \ldots, M\}$, we use a kernel $\hat{\mathbf{K}}^{(m)}$ of size $D_K \times D_K \times 1$. This kernel is convolved only with the corresponding channel of the input, producing an intermediate feature map $\hat{\mathbf{G}}$ of dimensions $D_F \times D_F \times M$.



■ **Note** — Notice that there is no N factor here because each input channel is processed independently with its own filter.

2.4.2 Pointwise Convolution

The depthwise convolution only performs spatial filtering; it does not combine information across channels. The pointwise convolution step addresses this by applying N filters of size $1 \times 1 \times M$ to the depthwise output $\hat{\mathbf{G}}$. Each 1×1 filter linearly combines the M channels at each spatial location, producing the final output feature map $\mathbf{G}$ of dimensions $D_F \times D_F \times N$.

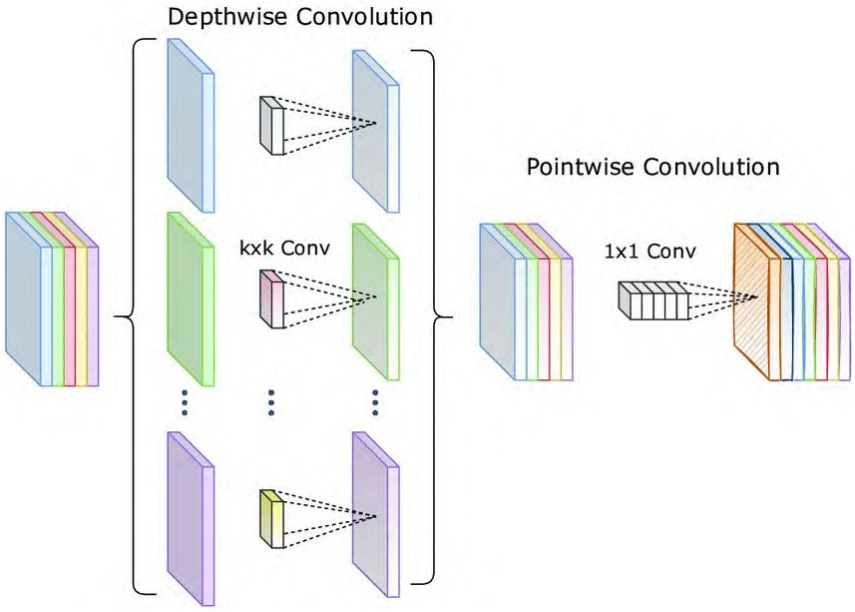


Figure 8: Depthwise separable convolution: depthwise convolution (left) applies a separate $k \times k$ filter to each input channel, performing spatial filtering without channel mixing. Pointwise convolution (right) then applies 1×1 filters to combine information across channels, producing the final output feature map. This factorization drastically reduces computational cost and parameter count compared to standard convolution.

2.5 Activation Functions in Convolutional Layers

Activation functions allow the network to learn complex patterns and decision boundaries.

2.5.1 Why Nonlinearity?

A convolutional layer (without activation) performs a linear operation. Activation functions break this linearity, enabling the network to approximate any continuous function (universal approximation theorem) and to learn hierarchical features.

2.5.2 Rectified Linear Unit (ReLU)

The most commonly used activation in modern CNNs is the **Rectified Linear Unit (ReLU)**, defined as:

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

ReLU has several desirable properties:

- **Computational efficiency**: ReLU involves only a simple thresholding operation ($\max(0, x)$), which is significantly faster to compute than exponentials or divisions required by sigmoid or tanh.
- **Sparsity**: By outputting exactly zero for all negative inputs, ReLU naturally creates sparse representations. This sparsity also acts as a form of implicit regularization.

- **Mitigation of the vanishing gradient problem**: Saturating activations whose gradients approach zero in the high-activation regime. This constant gradient allows error signals to propagate effectively through many layers, enabling the training of truly deep architectures.

3 Pooling Layers

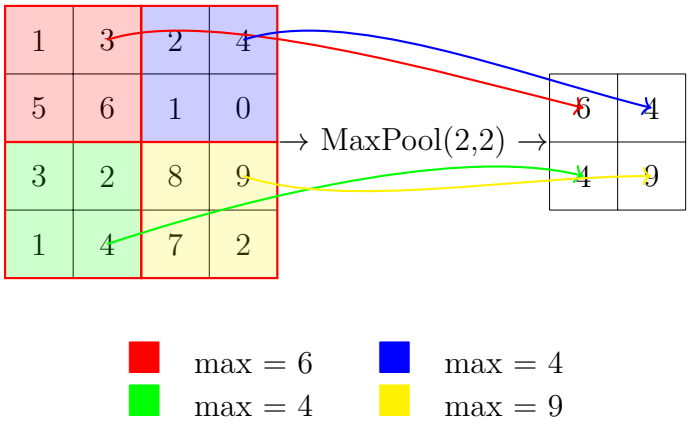
Pooling is a downsampling operation that reduces the spatial dimensions of feature maps, providing translation invariance and decreasing computational load. It operates independently on each input channel, typically using a fixed window (e.g., 2×2) and a stride (often equal to the window size).

Definition 3.1 : Pooling

Pooling reduces the spatial dimensions of feature maps, providing translation invariance and reducing computational load. It operates on each channel independently.

3.1 Max Pooling

Max pooling selects the maximum value from each pooling window. It preserves the most activated features and is the most widely used pooling variant. It helps retain prominent patterns while discarding less important information.





■ Note — **Max pooling** selects the **maximum** value from each pooling window, preserving the most activated features and is the most widely used variant. Other pooling types include average pooling (computes the mean), **min pooling** (takes the **minimum**), and **global pooling** variants (e.g., **global average** pooling) which downsample each feature map to a single value, often replacing fully connected layers to reduce parameters.

4 Flatten Operation

Its role is to transform multi-dimensional feature maps into a one-dimensional vector that can be processed by standard neural network layers.

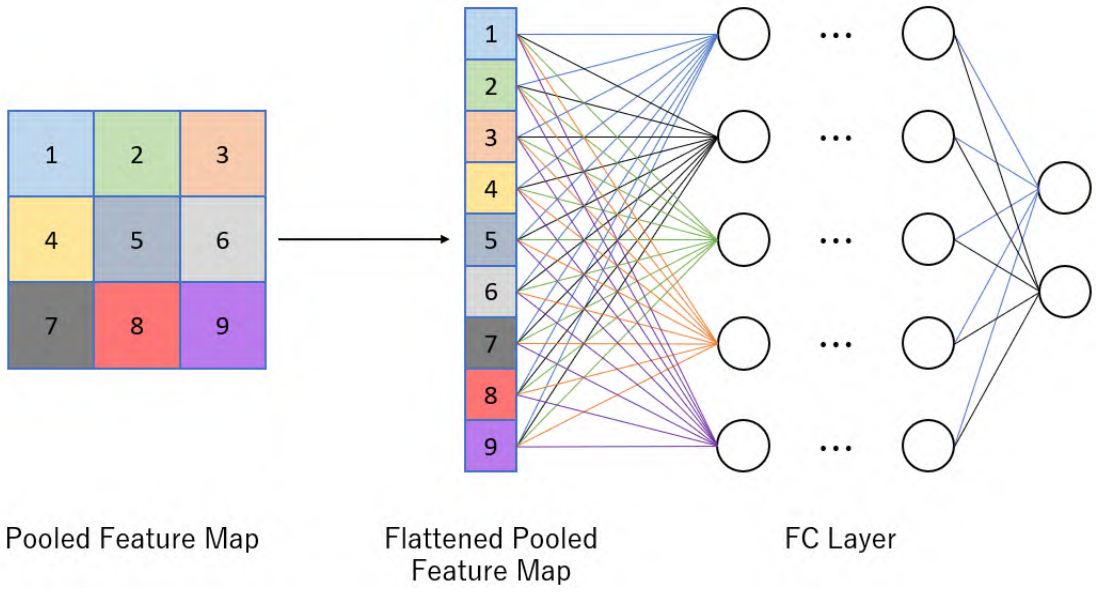


Figure 9: Flatten operation after pooling. The pooled feature map (e.g., 3×3) is flattened into a one-dimensional vector, which is then fed into a fully connected (FC) layer. The numbers illustrate the values being rearranged.

5 Fully Connected Layers

Fully connected (FC) layers, also known as dense layers, are a fundamental component of neural networks. In CNNs, they typically appear at the end of the architecture, after the convolutional and pooling stages, and serve as the decision-making part that maps the learned features to the final output (e.g., class probabilities).

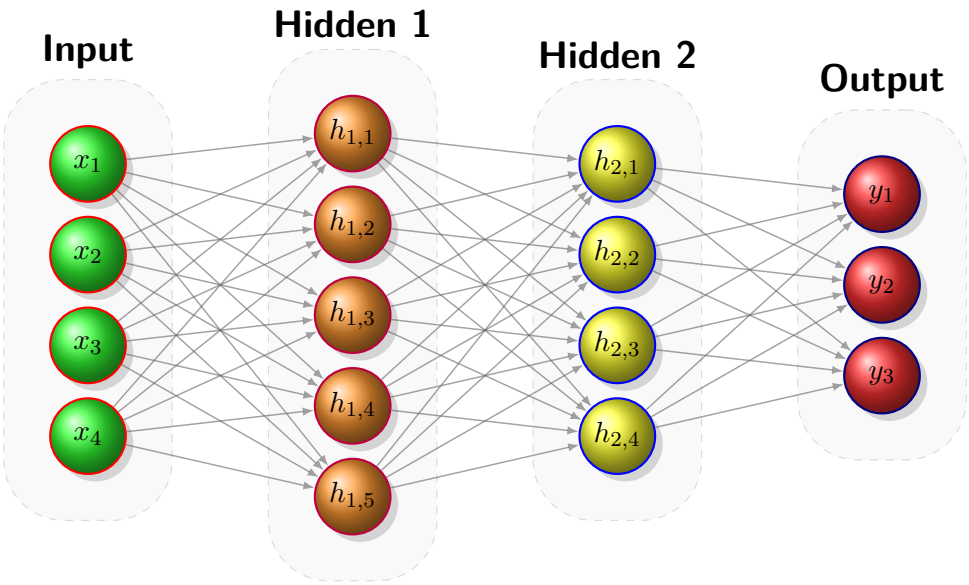


Figure 10: A fully connected layer: each input neuron is connected to every output neuron. The strength of each connection is determined by a learnable weight.

6 CNN Architecture Recap

A typical Convolutional Neural Network is built by stacking several fundamental operations in a sequential manner. The overall design follows a pattern that can be summarized in the following steps:

- **Convolution + ReLU**: A convolution layer extracts local features by sliding filters over the input, and the ReLU activation introduces non-linearity by setting negative values to zero. These two operations are often considered together as a single feature extraction unit.
- **Pooling**: A pooling layer (max, average, or min) downsamples the feature maps, reducing spatial dimensions and providing translation invariance. It is usually placed after one or more convolution+ReLU stages.
- **Repetition**: The combination of convolution+ReLU and pooling can be repeated multiple times, forming a deep hierarchy. As the network deepens, the number of filters increases while the spatial size decreases, capturing features at different scales.
- **Flatten**: After the final pooling stage, the multi-dimensional feature maps are converted into a one-dimensional vector. This operation preserves all extracted information but reorganizes it for the next step.
- **Fully Connected Layers**: The flattened vector is passed through one or more dense layers that perform high-level reasoning and produce the final output (e.g., class scores).

The diagram below illustrates this generic pipeline. The repeating nature of the convolutional stage is indicated by a dashed arrow and an ellipsis.

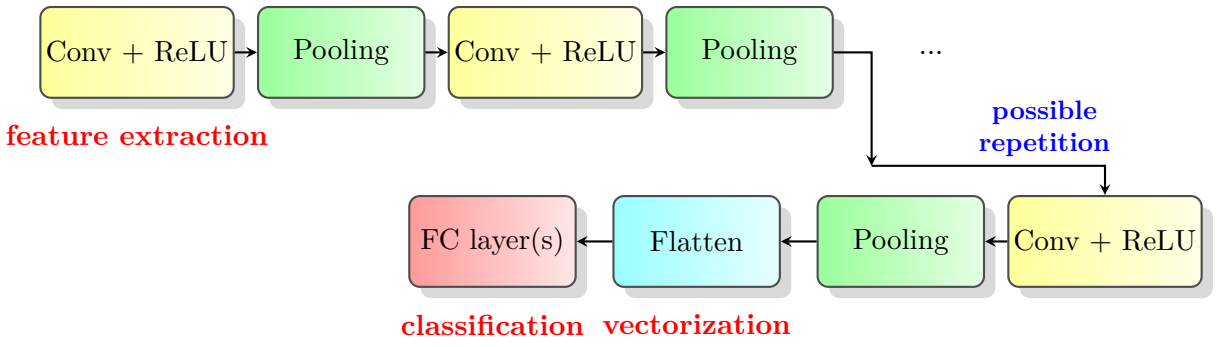


Figure 11: CNN architecture: repeated convolution+ReLU and pooling stages extract features, followed by flattening and fully connected layers for classification.

The following diagram provides a comprehensive overview of the standard architecture used to construct a Convolutional Neural Network (CNN) for image processing tasks. It visually breaks down the two primary stages of the model: feature extraction and classification. The diagram illustrates how the input image is processed through successive layers of convolution and pooling operations, activated by ReLU functions, to generate

feature maps. These extracted features are then flattened and passed through fully connected layers, culminating in a SoftMax function that produces a probabilistic output distribution, identifying the image's content.

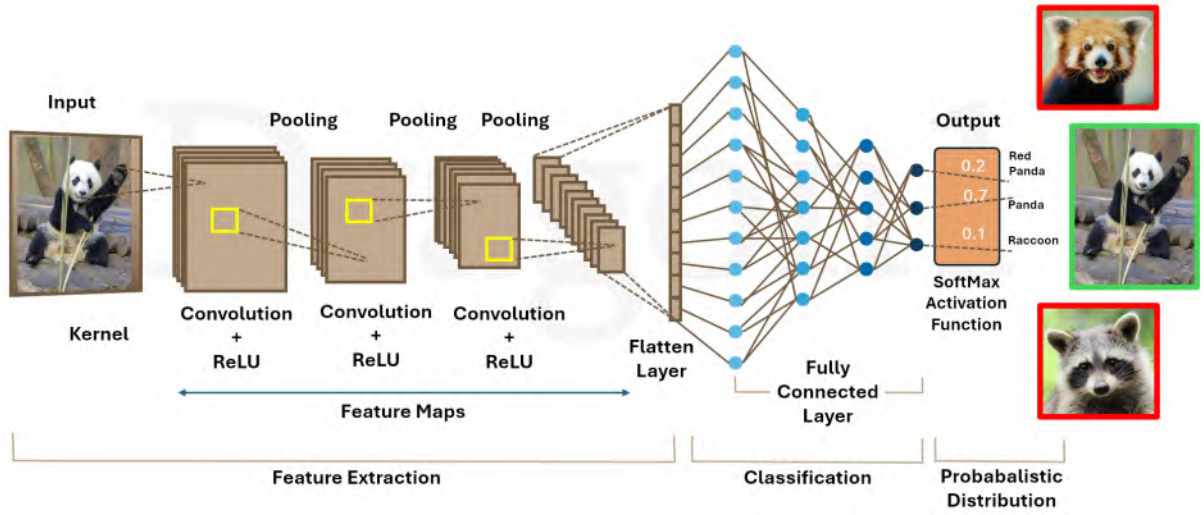


Figure 12: A diagram illustrating a typical Convolutional Neural Network (CNN) architecture for image classification. It shows the input image, followed by multiple stages of convolution (with ReLU activation) and pooling for feature extraction, then flattening and fully connected layers for classification, ending with a softmax output that produces a probabilistic distribution over classes (e.g., panda, raccoon). Key components such as kernels, feature maps, and the flow from feature extraction to classification are labeled.

7 Major CNN Architectures

7.1 Evolution of CNN Architectures

7.2 LeNet-5 (1998)

Yann LeCun et al. introduced LeNet-5 for handwritten digit recognition (MNIST). It established the modern CNN template: convolutional layers followed by pooling, then fully connected layers. LeNet-5 was deployed commercially for check reading.

- 7 layers (excluding input): 2 convolutional, 2 pooling, 3 fully connected
- Kernel sizes: 5x5, stride 1, pooling 2x2
- Activation: tanh/sigmoid
- Trained on MNIST, achieved 99.2% accuracy

Table 1: Major CNN Architectures (1998–2024)

Year	Architecture	Top-1 Acc. (ImageNet)	Key Innovation
1998	LeNet-5	–	First modern CNN
2012	AlexNet	63.3%	ReLU, Dropout, GPU
2014	VGGNet	74.4%	Very deep (16–19 layers), uniform 3x3 conv
2014	GoogLeNet (Inception v1)	74.8%	Inception modules, efficient computation
2015	ResNet	80.1%	Residual connections (152 layers)
2016	DenseNet	80.6%	Dense connectivity
2017	Xception	81.0%	Depthwise separable convolutions
2017	MobileNet	70.6%	Lightweight for mobile
2019	EfficientNet	84.4%	Neural architecture search + compound scaling
2021	ConvNeXt	87.8%	Modernized ResNet with Transformer ideas
2024	MaxViT	88.5%	Hybrid CNN-Transformer

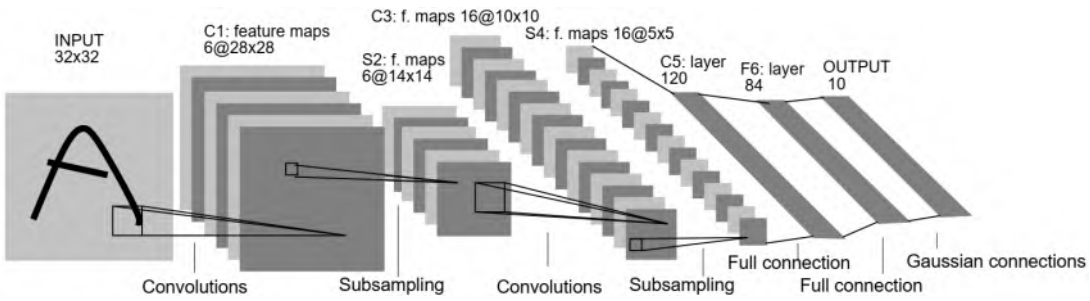


Figure 13: LeNet-5 architecture: two convolutional layers (C1, C3), two subsampling (pooling) layers (S2, S4), and three fully connected layers (C5, F6, output). Input is 32×32 grayscale.

7.3 AlexNet and the ImageNet Revolution (2012)

Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton achieved a dramatic breakthrough in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012, dropping the top-5 error rate from 25.8% to 16.4%. Key innovations:

- **ReLU activation**: solved vanishing gradient
- **Dropout**: reduced overfitting
- **GPU training**: 100x speedup using two GTX 580 GPUs
- **Data augmentation**: random cropping and horizontal flips
- 8 layers: 5 convolutional, 3 fully connected
- First use of ReLU, Dropout (0.5), and data augmentation
- Trained on two GPUs with model parallelism

- Input: $227 \times 227 \times 3$ (actually 224x224 with padding)
- Large kernels in first layer: 11x11, stride 4

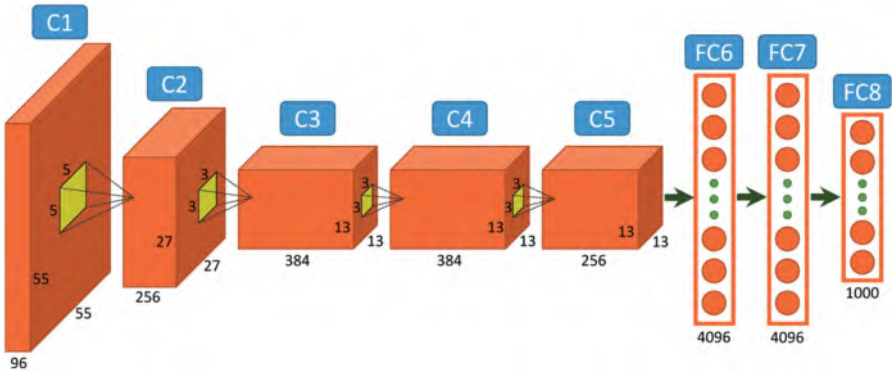


Figure 14: AlexNet architecture: 5 convolutional layers (some followed by max-pooling) and 3 fully connected layers. The network is split across two GPUs (shown as two streams).

7.4 VGGNet (2014)

- Very deep (16 or 19 layers) but simple: only 3x3 convolutions with stride 1, same padding, and 2x2 max pooling.
- Stack of small kernels increases receptive field with fewer parameters than one large kernel.
- Drawback: huge number of parameters (138M for VGG-16), computationally expensive.

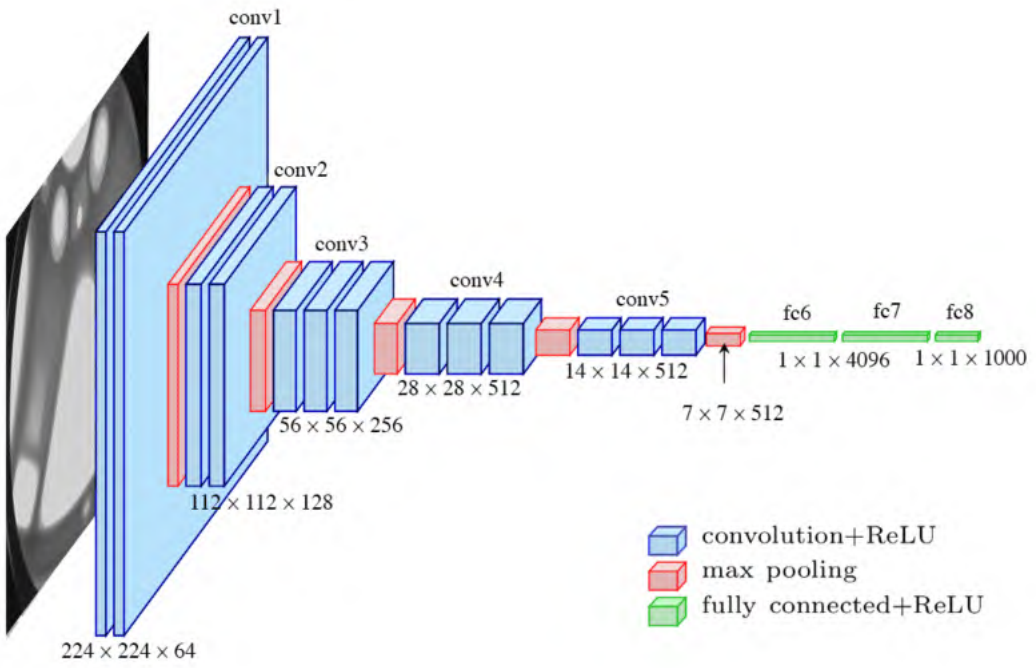


Figure 15: VGG-16 architecture: 13 convolutional layers (all 3×3, stride 1, same padding) and 3 fully connected layers. Pooling (2×2, stride 2) is applied after certain conv blocks.

7.5 GoogLeNet / Inception v1 (2014)

- Introduced **Inception modules**: parallel convolutions of different sizes (1x1, 3x3, 5x5) plus pooling, concatenated.
- 1x1 convolutions used for dimensionality reduction.
- Auxiliary classifiers to combat vanishing gradient.
- Only 4M parameters (much smaller than VGG).

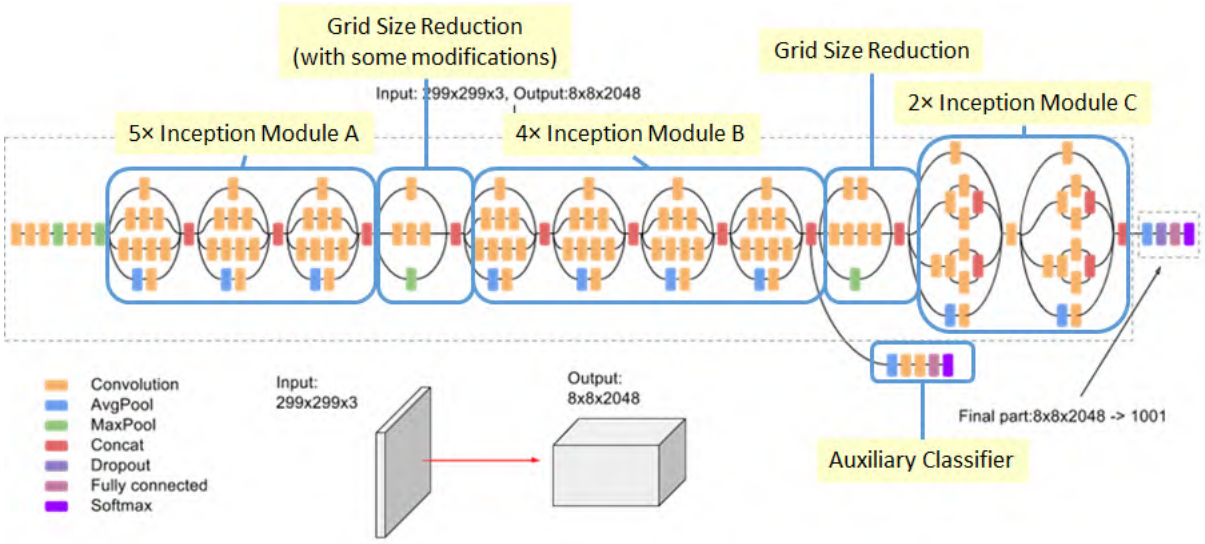


Figure 16: Inception module (naive version). Parallel convolutions (1×1, 3×3, 5×5) and 3×3 max pooling are concatenated along the channel dimension. In practice, 1×1 convolutions are added before the 3×3 and 5×5 to reduce dimensionality.

7.6 ResNet (2015)

- Introduced **residual connections** (skip connections) allowing training of very deep networks (up to 152 layers).
- The building block: $F(x) + x$, where $F(x)$ is a stack of convolutions.
- Solves the degradation problem (deeper networks not necessarily better).
- He initialization and batch normalization are essential.

7.6.1 ResNet-152 (152-layer version, 2015)

Main characteristics:

- Introduced in 2015 by Kaiming He et al. (Microsoft Research).
- Deep architecture with 152 layers using "bottleneck" residual blocks (1×1, 3×3, 1×1) to reduce the number of parameters.
- Residual connections enable stable training despite the depth.
- Pretrained on ImageNet (1.2 million images, 1000 classes).
- Top-1 accuracy: 77.0%, top-5: 93.3% on ImageNet.

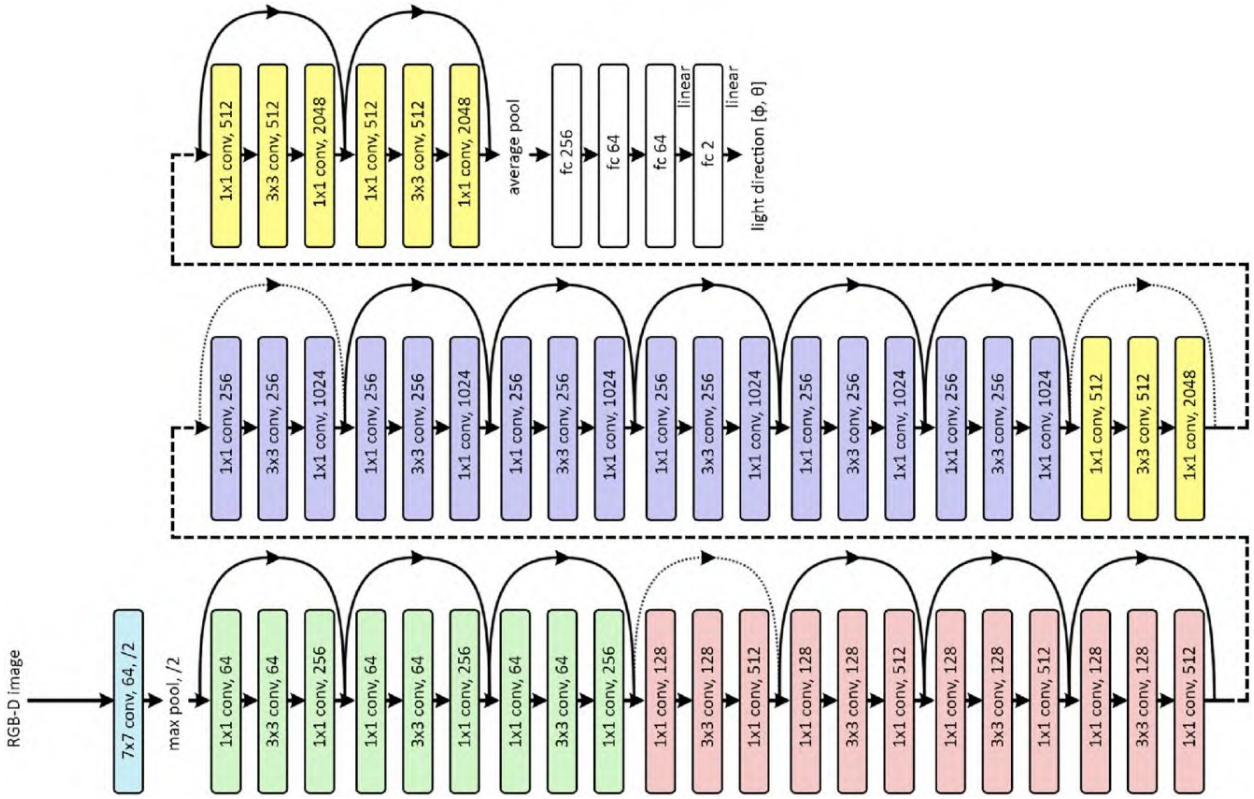


Figure 17: Residual block (alternative view): the input x is added to the output of two convolutional layers ($F(x)$). The skip connection allows gradients to flow directly through the block.

7.6.2 ResNet-50 (lightweight 50-layer version, 2015)

Main characteristics:

- Also introduced in 2015 by Microsoft Research.
- 50 layers with bottleneck architecture (fewer blocks than ResNet-152).
- 25.6 million parameters (about 2.4× fewer than ResNet-152).
- Top-1 accuracy: 76.0%, top-5: 93.0% on ImageNet.

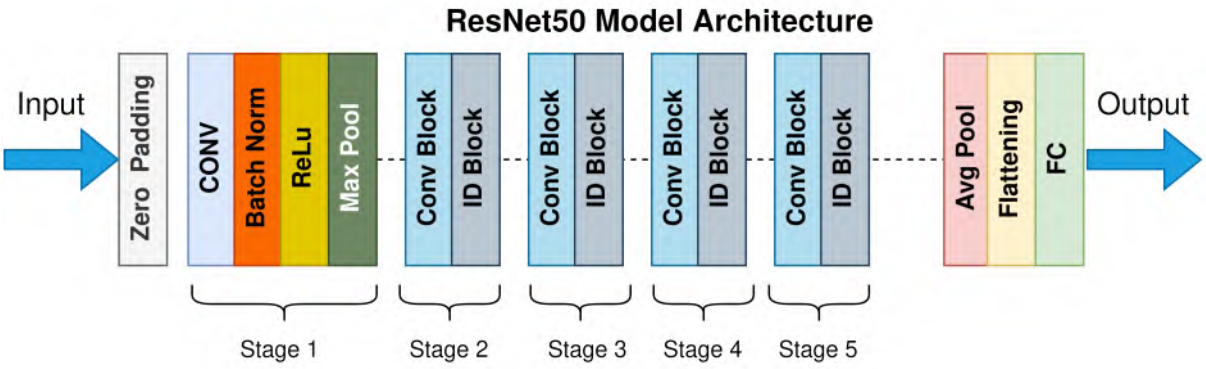


Figure 18: Residual block: the input x is added to the output of two convolutional layers ($F(x)$). The skip connection allows gradients to flow directly through the block.

Table 2: Comparison ResNet-152 vs ResNet-50

Feature	**ResNet-152**	**ResNet-50**
Year	2015	2015
Number of layers	152	50
Parameters (millions)	60.3	25.6
Top-1 accuracy (ImageNet)	77.0%	76.0%
Recommended use	Large datasets, complex tasks	Prototyping, perf/cost balance

7.7 DenseNet (2016)

- Each layer receives feature maps from all previous layers (dense connectivity).
- Encourages feature reuse, reduces number of parameters.
- Each layer adds only a few new feature maps (growth rate).

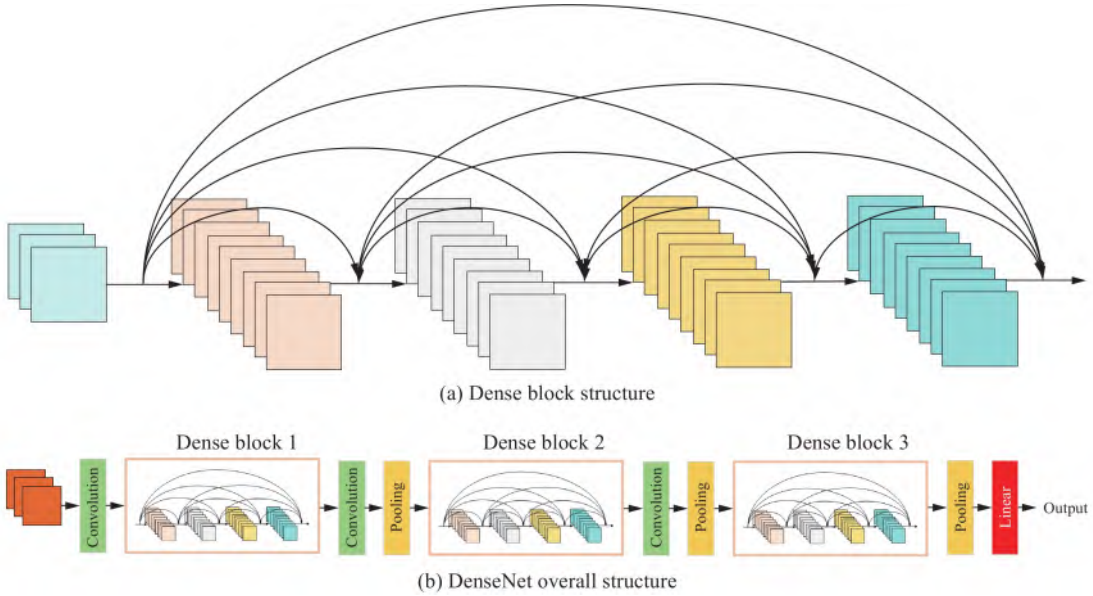


Figure 19: Illustration of DenseNet architecture. (a) A dense block structure: each layer receives concatenated inputs from all preceding layers, promoting feature reuse. (b) Overall DenseNet architecture composed of multiple dense blocks separated by transition layers (convolution and pooling).

7.8 MobileNet (2017)

- Uses **depthwise separable convolutions** to drastically reduce computation.
- Depthwise conv: each input channel convolved separately.
- Pointwise conv: 1x1 conv to combine channels.
- Suitable for mobile and embedded devices.

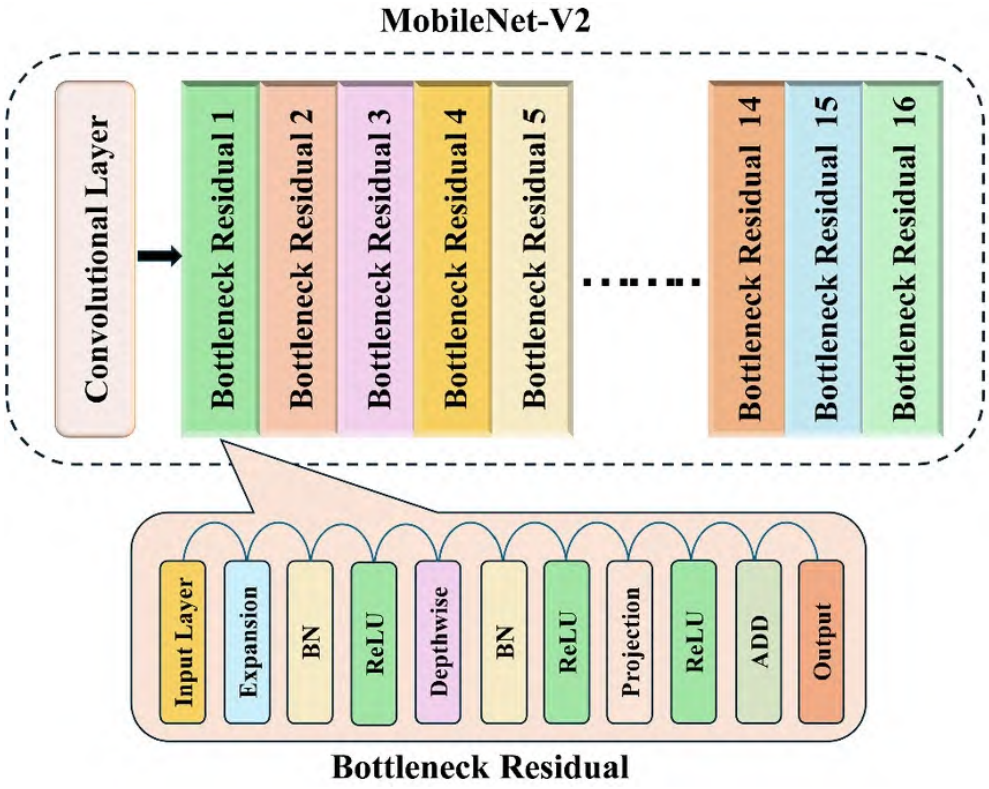


Figure 20: MobileNet-V2 architecture. The network consists of an initial convolutional layer (Conv1) followed by a sequence of 16 bottleneck residual blocks. Each bottleneck block follows an inverted residual structure: first an expansion layer (1×1 convolution) that increases the number of channels, then a depthwise convolution, and finally a projection layer (1×1 convolution) that reduces channels. Batch normalization and ReLU are applied after each convolution, except the projection which uses a linear activation. Skip connections (addition) are present when the stride is 1 and input/output dimensions match.

7.9 EfficientNet (2019)

- Uses neural architecture search (NAS) to find optimal scaling of depth, width, and resolution.
- Compound scaling: uniformly scale all dimensions with a compound coefficient.
- Achieves state-of-the-art accuracy with fewer parameters and FLOPS.

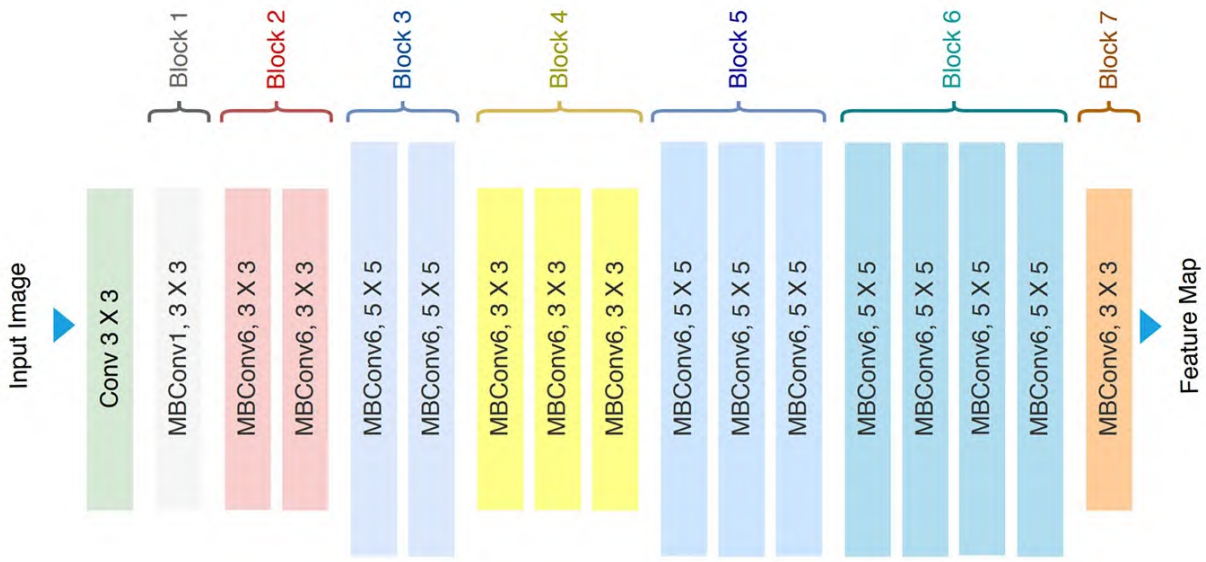


Figure 21: EfficientNet-B0 architecture. The network begins with a standard 3×3 convolution, followed by 7 mobile inverted bottleneck blocks (MBConv) with varying kernel sizes (3×3 and 5×5). Each MBConv block consists of an expansion layer, depthwise convolution, squeeze-and-excitation optimization, and a projection layer. The blocks are arranged in stages with increasing feature map depth and decreasing spatial resolution. This baseline architecture is optimized through compound scaling to create the EfficientNet family.

8 Hands-On Example: CNN for CIFAR-10 Classification

8.1 Dataset Description

CIFAR-10 consists of 60,000 32x32 color images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). 50,000 training, 10,000 test.

8.2 Data Loading and Preprocessing

```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import numpy as np

# Load CIFAR-10
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

# Normalize pixel values to [0,1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

```

```
# One-hot encode labels
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

8.3 Data Augmentation

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

8.4 Model Definition

```
def create_cnn():
    model = models.Sequential([
        layers.Input(shape=(32,32,3)),
        data_augmentation,
        layers.Conv2D(32, (3,3), activation='relu', padding='same'),
        layers.Conv2D(32, (3,3), activation='relu', padding='same'),
        layers.MaxPooling2D((2,2)),
        layers.Dropout(0.25),

        layers.Conv2D(64, (3,3), activation='relu', padding='same'),
        layers.Conv2D(64, (3,3), activation='relu', padding='same'),
        layers.MaxPooling2D((2,2)),
        layers.Dropout(0.25),

        layers.Conv2D(128, (3,3), activation='relu', padding='same'),
        layers.Conv2D(128, (3,3), activation='relu', padding='same'),
        layers.MaxPooling2D((2,2)),
        layers.Dropout(0.25),

        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')
    ])
    return model

model = create_cnn()
model.summary()
```

8.5 Compilation and Training

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint('best_cnn.h5', save_best_only=True)
]

history = model.fit(
    x_train, y_train,
    batch_size=64,
    epochs=50,
    validation_split=0.2,
    callbacks=callbacks
)
```

8.6 Evaluation

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
```

9 Applications of Convolutional Neural Networks

9.1 Object Detection

Three types of models are commonly distinguished for object recognition:

Table 3: Object Recognition Tasks

	Image classification	**Classification w. localization**	**Detection**
Description	Classifies a picture, predicts probability of object	Detects an object in a picture, predicts probability and location	Detects up to several objects in a picture, predicts probabilities and locations
Example model	Traditional CNN	Simplified YOLO, R-CNN	YOLO, R-CNN

9.2 Image Segmentation

Image segmentation assigns a class label to every pixel in an image. It is divided into semantic segmentation (classifying each pixel into a predefined category) and instance

segmentation (distinguishing individual objects within the same category). This is crucial in medical imaging, autonomous driving, and satellite image analysis.

Table 4: Image Segmentation Models

Model	Description	Use Cases
U-Net	Encoder-decoder architecture with skip connections; excels on biomedical images	Medical image segmentation
DeepLab	Uses atrous convolution and conditional random fields for precise boundary detection	Semantic segmentation in general scenes
Mask R-CNN	Extends Faster R-CNN by adding a branch for predicting segmentation masks	Instance segmentation

Conclusion

Convolutional Neural Networks have revolutionized computer vision and remain a fundamental tool in deep learning. Their core principles—sparse connectivity, parameter sharing, and hierarchical feature learning—have proven remarkably effective. From LeNet-5 to modern architectures like EfficientNet and ConvNeXt, CNNs continue to evolve, incorporating ideas from other domains (e.g., attention mechanisms).

In the next chapter, we will explore **Recurrent Neural Networks** and their variants for sequential data, building upon the concepts learned here.

"With great depth comes great representational power,
but also great responsibility in training."